Code & Conscience

DevOps Engineer — Practical Assignment

Multi-Cloud Cost Hygiene & Automation Challenge

Time budget	Target 6–10 hours of focused work, spread over up to 5 calendar days from the date you receive this brief.
Cost to you	Zero. The entire assignment runs locally using free, open-source tools (LocalStack, Moto, Docker, Terraform, GitHub Actions). No real cloud account or paid services required.
Submission	A single public GitHub repository link, structured exactly as described in Section 6. Late or differently-structured submissions will be deprioritized.
Use of AI	Permitted, with disclosure (see Section 7). We care about your judgment, debugging, and trade-offs — not your typing speed.

1. Context — Why we are giving you this assignment

Code & Conscience is launching a new business vertical focused on cost optimisation for cloud and AI infrastructure. As AI workloads explode and multi-cloud sprawl becomes the norm, our clients are bleeding money on idle GPUs, orphaned storage, untagged dev resources, and over-provisioned clusters. We are building a dedicated practice — a mix of consulting engagements and product-style automation — to help them find that waste, fix it, and prevent it from coming back. This role is one of the early DevOps hires for that vertical, and the work will sit squarely at the intersection of multi-cloud infrastructure, FinOps, and automation.

This assignment is a small but realistic slice of what the day-to-day looks like: provisioning multi-cloud infrastructure as code, building automation that detects and cleans up wasteful resources, and wiring up CI/CD so the cost hygiene is enforced continuously instead of as a one-off cleanup. It is intentionally not a textbook quiz — there is no single correct answer for several of the design choices, and we want to see how you think.

You are working with a fictional client, NimbusKart, an e-commerce startup running a small Python web service on AWS. NimbusKart's cloud bill quietly grew from ~$400/month to ~$2,100/month over the last quarter. Engineering suspects there are orphaned resources (unattached EBS volumes, stopped EC2 instances left running for weeks, unused Elastic IPs, old snapshots, untagged dev resources). Your job is to set up the foundation that lets them find, fix, and prevent this — using only local tooling so nobody runs up a real bill while you work.

2. What you will build (at a glance)

Three deliverables, in one repository. Each is small on its own; the value is in how they fit together.

Part	Deliverable	What it proves
A	Terraform stack for NimbusKart's baseline infra (VPC + EC2 + S3 + tagging policy), targeting LocalStack.	You can write Infrastructure as Code that is modular, applies cleanly, and follows tagging conventions — the foundation of every cost-attribution and cleanup workflow.
B	"Cost Janitor" — a Python or Bash script that detects orphaned AWS resources, plus a GitHub Actions workflow that runs it on every PR.	You can write practical automation that finds waste, integrate it into CI/CD, and reason about safe vs unsafe destructive actions.
C	A short design note (max 2 pages) on how you would harden, scale, and productionise this for a real multi-cloud setup.	You can think beyond a single script — about security, multi-account, multi-cloud, and where humans need to stay in the loop.

3. Part A — Infrastructure as Code (Terraform on LocalStack)

3.1 What to build

Write Terraform configuration that, when applied against LocalStack (https://localstack.cloud), creates the following resources for NimbusKart's staging environment:

- One VPC (10.20.0.0/16) with two public subnets in two AZs.
- One security group allowing inbound 80/443 from anywhere and 22 from a configurable CIDR (default 0.0.0.0/0 — but flag this; see 3.3).
- Two t3.micro EC2 instances tagged as a web tier.
- One S3 bucket for application logs, versioning enabled, with a lifecycle rule to expire non-current versions after 30 days.
- One additional EBS volume created but intentionally not attached to anything (we will use this in Part B as a known orphan).

3.2 Required structure & conventions

- Split your code into at least one reusable module (e.g. a network module). A flat main.tf with everything inline is a fail.
- Every resource that supports tags must carry, at minimum: Project, Environment, Owner, ManagedBy. ManagedBy must be "terraform".
- Use variables.tf with sensible defaults; do not hard-code region or environment names inside resources.
- Include an outputs.tf exposing the VPC ID, subnet IDs, and bucket name.
- Run terraform fmt and terraform validate before committing — both must pass cleanly.

3.3 Judgment calls we want to see

The spec above contains at least two things you should push back on if you were building this for real. We do not want you to silently do what we said — we want you to do what is right and explain the gap.

In your README, under a heading "Decisions & deviations", list every place where you (a) deviated from the spec, (b) followed the spec but think it is wrong, or (c) made an assumption because the spec was unclear. For each, give a one-sentence reason. This section is one of the most important parts of your submission.

3.4 How to test locally

LocalStack runs the AWS APIs locally inside Docker, so terraform apply works without a real AWS account.

```
# Start LocalStack
docker run --rm -d -p 4566:4566 --name localstack localstack/localstack

# Point Terraform at it via a provider block or tflocal wrapper
pip install terraform-local
tflocal init
tflocal apply -auto-approve
```

Your README must contain the exact commands a reviewer would copy-paste to reproduce your stack from a clean machine.

4. Part B — The "Cost Janitor" automation

4.1 The problem

NimbusKart wants a job that scans its AWS account daily and produces a list of resources that look wasteful. Build a script that does this against a LocalStack environment (or against Moto, if you prefer mocking at the SDK level).

4.2 Requirements

- Language: Python 3.10+ (preferred) or Bash. Pick one; do not mix.
- The script must detect at least these four orphan patterns:
  - EBS volumes in "available" state (not attached to any instance).
  - EC2 instances in "stopped" state for more than N days (N configurable, default 14).
  - Elastic IPs not associated with any instance.
  - Any resource missing one or more of the required tags (Project, Environment, Owner).
- Output must be a single report.json file in a documented schema, plus a human-readable Markdown summary.
- The script must support a --dry-run flag (default) and a --delete flag. In --delete mode it must still skip anything tagged Protected=true.
- It must exit with a non-zero status code if any orphans are found in --dry-run mode (so CI can fail).

4.3 CI/CD wiring

Create a GitHub Actions workflow (.github/workflows/cost-janitor.yml) that:

1. Spins up LocalStack as a service container.
2. Applies your Terraform from Part A against it.
3. Runs the Cost Janitor in --dry-run mode.
4. Uploads the report.json and Markdown summary as workflow artifacts.
5. Posts the Markdown summary as a comment on the PR if any orphans are found.

The workflow must run successfully end-to-end on a fresh fork — meaning we will fork your repo, open a dummy PR, and watch what happens. If the workflow does not run for us, we cannot evaluate this part.

4.4 Required output schema

Your report.json must conform exactly to this structure (fields may be added, but none of the listed fields may be renamed or removed):

```
{
  "scan_timestamp": "2026-01-15T10:00:00Z",
  "account_id": "000000000000",
  "region": "us-east-1",
  "summary": {
    "total_orphans": 5,
    "estimated_monthly_waste_usd": 42.50
  },
  "findings": [
    {
      "resource_id": "vol-abc123",
      "resource_type": "ebs_volume",
      "reason": "unattached",
      "age_days": 21,
```

```
      "estimated_monthly_cost_usd": 8.00,
      "tags": {"Project": null, "Environment": null},
      "suggested_action": "delete",
      "safe_to_auto_delete": false
    }
  ]
}
```

Cost estimates can use static per-unit prices hard-coded in a small constants file (e.g. EBS gp3 at $0.08/GB-month). Cite your source for any number you put in there.

5. Part C — Design note (max 2 pages)

Write a short document (Markdown, in the repo, named DESIGN.md) addressing the following. Be specific. Generic answers like "use IAM best practices" will not score.

- **Multi-cloud reality.** NimbusKart wants to add GCP next quarter. How would you restructure the Janitor so adding GCP (and later Azure) does not require rewriting the core? Sketch the module boundaries.
- **Permissions.** What IAM permissions does the Janitor need in --dry-run mode versus --delete mode? Write out the minimal IAM policy (JSON) for read-only mode.
- **Safety net.** Describe two specific failure modes where naïve auto-deletion could cause an outage at NimbusKart, and what guardrails you would add.
- **Observability.** Which 3–5 metrics would you publish (and to where) so the FinOps team can tell whether the Janitor is working? Name the metric, the source, and the threshold that should trigger an alert.
- **What you did not build.** One paragraph listing things you consciously left out and why. We are looking for honest scoping, not a feature list.

6. Submission instructions — read carefully

We receive many submissions for every role. To keep evaluation fair and fast, your repository must follow this exact structure. Submissions that ignore the structure will be reviewed last, if at all.

6.1 Repository layout

```
your-repo/
├── README.md                  <- entry point; see 6.2 for required sections
├── SUBMISSION.md              <- fixed-format index (see 6.3)
├── DESIGN.md                  <- Part C
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── modules/
│       └── network/
├── janitor/
│   ├── janitor.py (or janitor.sh)
```

```
│   ├── requirements.txt
│   ├── constants.py           <- pricing constants with sources
│   └── tests/
├── .github/
│   └── workflows/
│       └── cost-janitor.yml
├── docs/
│   └── walkthrough.md         <- link to your video + transcript
└── samples/
    ├── report.example.json    <- a sample output file
    └── report.example.md
```

6.2 README must contain these sections, in this order, with these exact headings

- ## Overview — one paragraph in your own words.
- ## How to run locally — copy-pasteable commands, starting from `git clone`.
- ## Architecture — one diagram (PNG, SVG, or ASCII).
- ## Decisions & deviations — bulleted list, one line each (see 3.3).
- ## Trade-offs — what you would do with one more week.
- ## AI usage disclosure — see Section 7.

6.3 SUBMISSION.md — fixed format

Copy the template below verbatim into a file called SUBMISSION.md at the repo root, and fill it in. We use this to locate everything quickly. Do not change the headings, the order, or the checkboxes.

```
# Submission — DevOps Engineer Assignment

**Candidate name:**
**Email:**
**Date submitted:**
**Hours spent (approximate):**

## Deliverables checklist
- [ ] Part A: Terraform code under /terraform applies cleanly on LocalStack
- [ ] Part A: `terraform validate` and `terraform fmt -check` both pass
- [ ] Part B: Janitor script runs in --dry-run mode and produces report.json
- [ ] Part B: GitHub Actions workflow runs green on a fresh PR
- [ ] Part B: --delete mode respects Protected=true tag
- [ ] Part C: DESIGN.md is present and within 2 pages
- [ ] Walkthrough video link below is accessible (unlisted is fine)

## Walkthrough video
Link (Loom / YouTube unlisted / Google Drive):
Length: max 5 minutes

## Sample report
Path to a sample report.json produced by your script:

## Known limitations
(bullet list — be honest)

## AI usage disclosure
(see Section 7 of the brief)
```

6.4 The walkthrough video (mandatory)

Record a screen capture (max 5 minutes, Loom or YouTube unlisted) where you: (a) start LocalStack and apply your Terraform live, (b) run the Janitor and walk through one finding in the report, (c) point to one design decision in your code you are proud of, and (d) point to one thing you would change. This is not a polished marketing video — talking head not required, mistakes are fine. We use it to confirm the code is genuinely yours and to hear how you talk about it.

6.5 How to send

Reply to the email/Whatsapp thread you received this assignment in, with:

- Public GitHub repository URL.
- Walkthrough video URL.
- One-line subject change: "[DevOps Assignment] <your full name>"

7. Use of AI tools

You may use ChatGPT, Claude, Copilot, Cursor, or any other AI assistant. Pretending you did not is the wrong answer. In your README's AI usage disclosure section, briefly cover:

- Which tools you used and roughly for what (e.g. "Copilot for boilerplate Terraform, ChatGPT to debug a moto error").
- One specific thing the AI got wrong or suggested badly, and how you noticed.
- One section of the code you wrote without AI help — and why you chose to do that part manually.

We are not testing whether you can avoid AI — we are testing whether you have the judgment to use it well. Submissions where every commit looks AI-generated and the candidate cannot explain their own code in the walkthrough video will be rejected.

8. How we evaluate

Each submission is reviewed against the rubric below. Total score out of 100.

Dimension	Weight	What we look for
Terraform correctness & structure	20	Code applies on LocalStack, uses modules, passes fmt/validate, follows tagging convention.
Janitor script quality	20	Detects all four orphan types, schema-compliant report, safe destructive behavior, sensible code structure.

CI/CD pipeline	15	Workflow runs green end-to-end on a fresh fork, artifacts uploaded, PR comment works.
Design note (Part C)	15	Specific, opinionated, shows multi-cloud thinking, names real failure modes.
Documentation & reproducibility	10	README runs us from clone to result without a single guess; SUBMISSION.md complete.
Decisions & deviations section	10	Candidate spotted the unsafe defaults and ambiguous parts of the spec, and made deliberate, defensible calls.
Walkthrough video	10	Candidate can explain their own code, points to a genuine trade-off, talks like an engineer.

8.1 Automatic disqualifiers

- Terraform code that does not even initialize against LocalStack.
- Janitor script that crashes on first run with default flags.
- No SUBMISSION.md, or SUBMISSION.md with the wrong structure.
- No walkthrough video, or a video where the candidate cannot answer basic questions about their own code.
- Git history showing the entire project committed in 1–2 commits (we look at commit cadence).
- Copy-pasted code from a public GitHub project without attribution. Inspired-by is fine; identical is not.

9. FAQ

Q. I do not have an AWS account. Can I still do this?

Yes. The entire assignment is designed for LocalStack + Docker on your laptop. You will not need any real cloud credentials.

Q. Can I use a different IaC tool (Pulumi, CDK)?

No. Use Terraform. We need to compare submissions on a common substrate.

Q. Can I use Python frameworks like boto3, click, rich?

Yes. Pin versions in requirements.txt.

Q. The spec for Part A says 0.0.0.0/0 on port 22 — that is unsafe. Should I do it anyway?

Re-read Section 3.3.

Q. What if I cannot finish everything?

Submit what you have, mark unfinished items as such in SUBMISSION.md, and be honest in the walkthrough. A partial, honest submission beats an inflated one.

Q. Who do I ask if I am stuck on the brief itself?

Email the recruiter who sent you this brief. Treat ambiguity in the spec the same way you would in a real ticket — make a call, document it under "Decisions & deviations", and move on.

Good luck — we are excited to see what you build.